<!-- Extracted from https://oracc.museum.upenn.edu/doc/help/lemmatising/primer/index.html on 2026-10-16 by a stdlib HTML->Markdown converter; see raw .html for the original. -->

Lemmatisation primer

▼ Contents

Lemmatization 1
- First Steps 2
- Initial Document 2
- Replacing X's 3
- New Forms 6
- Proper Nouns 7
- POS only 7
- Explicit 8
- SENSE and EPOS 9
- Morphology 11
- Checking your lemmatization 11

Next Steps 11

#lem: lines 12
- Separator 12
- Ambiguity 12
- Multiplexes 12
- Uncertainty 13
- Breakage 13
- Numbers 13
- Miscellanea 13

Units 13

Dictionaries 14

This page provides an introduction to linguistic annotation facilities, especially lemmatization, used in Oracc.

If you are not yet familiar with the concept and details of ATF please read the ATF Primer first.

Lemmatization

Lemmatization is the simplest and most common annotation which consists of labelling written words, which may be inflected, with the base word (or dictionary headword) of which the written form is an instance.

First Steps

So, you have an ATF file and you want to lemmatize it. When we say in this document "now lemmatize the text (again)", we mean: Use the Emacs interface; open the file and choose 'Lemmatize' from the ATF menu.

The lemmatizer is actually built in to the same ATF processor which does the checking.

We'll go through the procedure for creating a new lemmatization of an Akkadian text here, assuming that there is no dictionary or project glossary to rely on. In real life, projects will build glossaries as they go, and the ePSD and ePAD (electronic Pennsylvania Sumerian/Akkadian Dictionaries) will also help by enabling the lemmatizer to make better initial guesses. Here, though, we'll assume that the lemmatizer has no external information to help it, and we'll show how to build up the information which can be harvested to generate a simple glossary automatically.

In real life, too, we recommend that you translate and lemmatize each text at the same time: thinking about the two together significantly improves the quality of both. However, here we shall just focus on the process of lemmatization.

Initial Document

Let's say your initial document looks like this (OK, admittedly this is somewhat contrived, but it will serve to make the points we need to make):

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-x-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
2. A{+e} ba-[...]
3. 1 SILA₃ me-e {m}ri-hat-{d}60
4. mu-u
@reverse
$ reverse missing
```

Now, lemmatize that file and you should see this:

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-x-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
#lem: X

2. A{+e} ba-[...]
#lem: X; u

3. 1 SILA₃ me-e {m}ri-hat-{d}60
#lem: n; X; X; X

4. mu-u
#lem: X

@reverse
$ reverse missing
```

The lemmatizer has inserted interlinear lemmatization lines and carried out a minimal preliminary analysis of the forms, sorting them into numbers (`n` in line 3); unlemmatizable forms (`u` in line 2); and unknown forms (all those `X`'s).

Replacing X's

In this bottom-up model we are defining relations between forms (the words-between-spaces in the transliteration) and glossary items. In all cases, we prefix a lemmatization which contains any new information with a plus sign. The plus sign always means "I know that this form may not pass the ATF checker's validation of lemmata content; don't warn me about it; I'm adding information that can be harvested later for a glossary or dictionary."

A new lemmatization requires certain basic information:

The Citation Form (CF)
: The dictionary headword; the fundamental portion of the lemmatization.

The Guide Word (GW)
: The disambiguator used in the dictionary; in conventional dictionaries this is usually a number or letter, but the Corpus-Based Dictionary system uses a guide word which is normally a basic meaning of the word, or a hypernym (a superset designator such as "official").

The Sense (SENSE)
: An optional meaning, or sense, which a word has in the current context; it may be used as well as, or instead of, a GW.

The Part-of-Speech (POS)
: The part-of-speech; a complete list for any given language is available in the language-specific linguistic annotation documents, e.g., the Akkadian-specific documentation.

The Effective Part-of-Speech (EPOS)
: An optional part-of-speech tied to the current syntactic context.

Normalization (NORM0)
: The normalization for the form; not needed for Sumerian.

The Base (BASE)
: The written base of the form; only needed for Sumerian

The Morphology (MORPH)
: The morphological information for the form; not yet used with all languages.

Minimally, then, the first X in the sample file can be replaced with the lemmatization `+mû[water]N$`. In this example, the Citation Form (CF) precedes the Guide Word (GW), which is given in square brackets. Next comes the Part of Speech (POS), followed by a probably mysterious dollar symbol (`$`). This symbol introduces the normalized form of the word (NORM0); in Akkadian, the normalization is required for new definitions of lemmata. The form given here is abbreviated--the unadorned dollar symbol means that the normalized form is identical to the CF.

So, we now have:

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-x-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
#lem: +mû[water]N$

2. A{+e} ba-[...]
#lem: X; u

3. 1 SILA₃ me-e {m}ri-hat--{d}60
#lem: n; X; X; X

4. mu-u
#lem: X

@reverse
$ reverse missing
```

Lemmatize that file again, and you'll see this:

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
#lem: +mû[water]N$

2. A{+e} ba-[...]
#lem: X; u

3. 1 SILA₃ me-e {m}ri-hat--{d}60
#lem: n; X; X; X

4. mu-u
#lem: mû[water]N

@reverse
$ reverse missing
```

Note how the lemmatizer has learned the form `mu-u` and lemmatized the form in line 4 automatically. This shorter form of lemmatization comprises just the NORM0, the GW, and the POS.

Sometimes you will want to define a new lemma but indicate that the current context uses a sense other than the basic sense. You can do this by putting both the GW and the sense in square brackets, separated by double-slash (`//`), e.g.: `+awātu[word//command]N$`

New Forms

Often, you will have a new form of a lemma you have already defined. You need to tell the lemmatizer that there is novel information here by prefixing the lemmatization with a plus, as usual. For Akkadian you also need to give the normalization explicitly using the dollar convention if the normalization is new. (You can always try adding less information in the examples that follow to experience the many and varied complaints the ATF checker and lemmatizer generate when they don't have enough information.)

The next step, then is to lemmatize the forms of *mû* and *qû* in lines 2 and 3:

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-x-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
#lem: +mû[water]N$

2. A{+e} ba-[...]
#lem: +mû[water]N$mê; u

3. 1 SILA₃ me-e {m}ri-hat--{d}60
#lem: n; +qû[unit]N$qā; +mû[water]N$mê; X

4. mu-u
#lem: mû[water]N

@reverse
$ reverse missing
```

In short, whenever you want the lemmatizer to learn new information -- a new form or SENSE or NORM0 -- you must use the long form of lemmatization, `+CF[GW//SENSE]POS'EPOS$NORM0`. Lemmatizations added automatically (which match data already in the glossary) used the short form `NORM0[SENSE](E)POS`. (We will come back to SENSE and EPOS very shortly.)

Proper Nouns

Proper nouns can be lemmatized in either of two ways; an individual text can use a mixture if desired.

POS only

Lemmatizing proper nouns only by their POS is appropriate if lemmatization is being carried out in multiple phases--first the lexical information and later the proper nouns. Under this approach, a personal name is lemmatized simply as `PN`; a month name as `MN`, and so on. A list of POS

tags is available in the proper nouns linguistic annotation page. In our example, we would complete the lemmatization like this:

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-x-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
#lem: +mû[water]N$

2. A{+e} ba-[...]
#lem: +mû[water]N$mê; u

3. 1 SILA₃ me-e {m}ri-hat--{d}60
#lem: n; +qû[unit]N$qā; +mû[water]N$mê; PN

4. mu-u
#lem: mû[water]N

@reverse
$ reverse missing
```

Explicit

If you choose to lemmatization proper nouns explicitly, you need to give at least a CF and the POS, in which case you can leave the square brackets empty: `+Rihat-Anu[]PN$`

Note that the CFs of proper nouns do not mark vowel length.

In our example, we would have:

```
&P363704 = TCL 06, 32
#project: cams
#atf: lang akk-x-stdbab
#atf: use unicode

@tablet
@obverse
1. mu-u
#lem: +mû[water]N$

2. A{+e} ba-[...]
#lem: +mû[water]N$mê; u

3. 1 SILA₃ me-e {m}ri-hat--{d}60
#lem: n; +qû[unit]N$qā; +mû[water]N$mê; +Rihat-Anu[]PN$

4. mu-u
#lem: mû[water]N

@reverse
$ reverse missing
```

For most classes of proper nouns this approach is adequate--most divine names have only one possible referent, as do most geographical names. For personal names the situation is more complicated as the same name may frequently refer to different people. In such cases, the present solution is simply to put a number in the square brackets--a future implementation of the lemmatizer will support giving ancillary information such as a parent's name to serve as a disambiguator. Thus, `Dada[1]PN` and `Dada[2]PN` reference different entries in the prosopography.

SENSE and EPOS

Compare these two phrases in Akkadian: *ina mahar ili* and *mahar ili*. We might well translate both of them as "in front of the god" or "before the god" interchangeably, but there are important grammatical differences between the two. Compare these lemmatisations:

```
1. ina ma-har DINGIR
#lem: +ina[in]PRP$; +mahru[front]N$mahar; +ilu[god]N$ili

2. ma-har DINGIR
#lem: +mahru[front//before]N'PRP$mahar; ili[god]N
```

In the first line *mahar* is a construct-state noun, "front", following a preposition; in the second it *is* a preposition, "before". The lemmatization explicitly shows the context-dependent SENSE, separated from the GW with `//` but contained in the square brackets with it. The context-dependent EPOS (Effective Part-of-Speech) is marked with `'` immediately following the POS.

Running the lemmatizer again on a further instance of the same spelling will generate just the SENSE and the EPOS, not the GW and POS. EPOS always appear with the `'` preceding them, but auto-lemmatized SENSEs look just like GWs:

```
1. ina ma-har DINGIR
#lem: +ina[in]PRP$; +mahru[front]N$mahar; +ilu[god]N$ili

2. ma-har DINGIR
#lem: +mahru[front//before]N'PRP$mahar; ili[god]N

3. ma-har LUGAL
#lem: mahar[before]'PRP; +šarru[king]N$šarri
```

Every time you specify SENSE you need to specify an EPOS too, even if it is the same as the POS. For instance:

```
1. ina ma-har DINGIR
#lem: +ina[in]PRP$; +mahru[front//presence]N'N$mahar; +ilu[god]N$ili
```

The checker flags a SENSE without an EPOS as an error.

EPOS can also be useful on its own, however. For instance, with proper nouns:

```
4. {mul}GU.LA u {d}ŠUL.GI
#lem: +Gula[]DN'CN$; u[and]CNJ; +Šulgi[]RN'DN$
```

For more information on SENSE and EPOS in Akkadian, see the page on Akkadian linguistic annotation.

Morphology

If your project is specifying the morphology for each form, you should also give that information when you specify a new form. The morphology is given after a pound sign (`#`), and the content depends on the language. The special symbol tilde (`~`) indicates that the morphology is the base or uninflected form. Let's say there are morphology specifiers 'Sg.' for singular, '3' for second person and 'fem' for feminine. A morphology example might then look like this:

```
5. ta-ra-am
#lem: +râmu[love]V$tarām#3.Sg.fem
```

There is much more detail on annotating Sumerian morphology on the Sumerian linguistic annotation page.

Checking your lemmatization

After lemmatizing, you can review the ATF files to check ambiguous lemmatizations and typographical errors. If you are using Emacs, choose the Harvest Notices item on the ATF menu to generate a list of all the long-form lemmatizations (the ones with a plus sign) so that you can review them individually.

New lemmatizations should also be checked by project managers as part of the glossary management process. This is described in the documentation on Project Management with Emacs.

Next Steps

You should now be ready to start lemmatizing files; as you get more experience and have more questions you should refer to the language-specific pages on lemmatization:

- Akkadian
- Aramaic
- Elamite

- Greek
- Old Persian
- Sumerian
- Ugaritic

and, for all languages, the page on lemmatizing:

- proper nouns.

#lem: lines

This section gives a more formal description of the components of #lem: lines.

Separator

The sequence '`; `', i.e., semi-colon followed by space, is reserved as the separator between lemmatizations. There must be the same number of lemmatizations in the `#lem:` line as there are forms in the corresponding line of transliteration; the ATF processor signals an error when it detects mismatches of this kind. Special provision is made for preserving this 1:1 relationship when labelling broken forms or breakage on manuscripts as described below.

Ambiguity

Ambiguous forms may have multiple lemmatizations attached to them with the lemmatizations separated by vertical bars:

```
1. an-na
#lem: DN|an[sky]
```

The sequences either side of vertical bars are complete lemmatizations in their own right and may therefore have their own POS, morphology, disambiguation and any other characteristics.

Multiplexes

There are several circumstances in which a single orthorgraphic form ("word") actually writes more than one lemma: these include crasis and sandhi writings as well as logograms which are best treated as a single word (perhaps because of word order) but which correspond to more than one word in the target language (e.g., the writing `{d}UTU.E₃` for Akkadian `ṣīt šamši` "sunrise").

In all these cases, the input is analogous to the ambiguous forms described above, but the `&` is used instead of the vertical bar. Thus, `{d}UTU.E3` would be lemmatized as `ṣīt[exit]&šamši[sun]`. (Note, by the way, that compound phrases are always lemmatized according to their

constituents).

See the Akkadian linguistic annotation page for more details.

Uncertainty

Uncertainty in lemmatization is indicated by the use of the conventional lemmatization `X` (uppercase 'X'). This should be used when the form is in principle open to lemmatization but no lemmatization can be suggested.

Breakage

Breakage in the manuscript is lemmatized with the conventional lemmatization `u`; such forms are considered unlemmatizable.

Numbers

Numbers are lemmatized with the conventional lemmatization `n`; a special-purpose processor is planned for higher order annotation and manipulation of numerical data.

N.B. In narrative context, numbers should be lemmatized as words; in administrative context, the `n` convention should be used.

Miscellanea

The conventional lemmatization `M` is used where the form is a standalone instance of a morpheme such as occur in certain Mesopotamian lexical lists.

The conventional lemmatization `L` is used where the form is in a language that is not currently handled by the lemmatization system.

Units

Top-level unit (normally main sentence) boundaries can be annotated within the lemmatization by use of two conventions:

```
+. = insert unit boundary
-. = suppress unit boundary
```

The `+.` convention is relevant to all languages. It must occur either at the very beginning or the very end of the lemmatization string: if it precedes the lemmatization it must be followed by a space; it if follows the lemmatization it must be preceded by a space.

For some languages (e.g., Sumerian) most unit boundaries are correctly identified programmatically; where the program is wrong, the `-.` can be used to suppress a break. The `-.` convention is subject to the same rules for placement and whitespace as `+.`.

```
6. mu-na-du3
#lem: du[build] +.

...

10. e2 mu-na-du3 lugal-e
#lem: e[house]; du[build] -.; lugal[king] +.
```

Dictionaries

A specific type of dictionary, the Corpus-Based Dictionary XML datatype, is used by Oracc annotation to provide control lists of permitted CFs, GWs, Senses and POS information. Documentation of this format is in preparation.

This dictionary is the means of supplying POS information when it is not given explicitly (if given explicitly, the POS in the lemmatization overrides the one given in the dictionary).

The dictionary is also the means of canonicalizing lemmatizations of the form `CF[SENSE]` since such pairs can be looked up and the corresponding unique `CF[GW]` identified; this is relevant in the construction of forms files.
18 Dec 2019 osc at oracc dot org

Steve Tinney & Eleanor Robson

Steve Tinney & Eleanor Robson, 'Lemmatisation primer', *Oracc: The Open Richly Annotated Cuneiform Corpus*, Oracc, 2019 [http://oracc.museum.upenn.edu/doc/help/lemmatising/primer/]

Back to top ^^

Released under a Creative Commons Attribution Share-Alike license 3.0, 2014.
[http://www.facebook.com/opencuneiform] [http://oracc.blogspot.com]
[http://www.twitter.com/oracctivity]

Oracc uses cookies only to collect Google Analytics data. Read more here; see the stats here [http://www.seethestats.com/site/oracc.museum.upenn.edu]; opt out here.

http://oracc.museum.upenn.edu/doc/help/lemmatising/primer/